

Diseño de Software de Dispositivos Móviles

Unidad 7

Integración de Aplicaciones Móviles con Servicios

Backends, APIs RESTful y GraphQL, autenticación, almacenamiento en la nube y seguridad

Asignatura	Diseño de Software de Dispositivos Móviles
Carrera	Tecnicatura Universitaria en Diseño de Software - FTyCA, UNCA
Docente	Mg. Ing. Carlos Acosta Parra
Actividad asociada	Integrar autenticación o servicio externo (API REST / GraphQL / almacenamiento)
Entregable	Documento técnico explicando la integración + video demo mostrando la funcionalidad

Apuntes de cátedra · 2026

Objetivos de aprendizaje

Al finalizar esta unidad, el estudiante será capaz de:

1. Explicar el rol de los servicios backend en una aplicación móvil moderna y reconocer sus responsabilidades principales.
2. Seleccionar una estrategia de integración adecuada según el alcance, el equipo, la seguridad requerida y la evolución esperada del producto.
3. Consumir servicios RESTful desde una app móvil contemplando headers, códigos de estado, paginación, latencia, errores y datos incompletos.
4. Comprender cuándo GraphQL puede simplificar o complejizar el consumo de datos en pantallas móviles.
5. Distinguir autenticación y autorización, e implementar flujos básicos de sesión con tokens y almacenamiento seguro.
6. Gestionar datos y archivos en la nube considerando metadatos, permisos, URLs firmadas, sincronización y estados offline.
7. Aplicar buenas prácticas de seguridad en la integración: transporte seguro, validación en backend, protección de secretos, mínimo privilegio y manejo responsable de errores.
8. Documentar técnicamente una integración y presentarla mediante una demo funcional breve, clara y verificable.

Introducción

En la unidad 6 se trabajó el paso desde el prototipo hacia una aplicación funcional: pantallas implementadas, navegación real, gestión de estado, eventos, consumo de una API pública y validación básica del comportamiento. La unidad 7 profundiza ese camino: una app móvil deja de ser solamente una interfaz que consulta datos abiertos y empieza a formar parte de un sistema distribuido.

En una aplicación real aparecen usuarios identificados, permisos, datos propios, archivos, reglas de negocio, servicios externos, auditoría y decisiones de seguridad. El teléfono ya no es el único lugar donde vive la solución: la experiencia móvil depende de contratos con servidores, plataformas de backend como servicio, bases de datos, almacenamiento en la nube, proveedores de identidad y APIs de terceros.

Integrar no significa solamente “hacer una llamada HTTP”. Significa diseñar cómo viajan los datos, quién puede acceder a ellos, dónde se validan las reglas, cómo se recupera la app ante errores, qué se guarda localmente, qué se sincroniza, qué se registra para diagnosticar fallos y qué información nunca debería quedar expuesta en el dispositivo.

Idea central. Una integración móvil bien diseñada es un contrato entre la app, el backend, la identidad del usuario, los datos y las reglas de seguridad. Si ese contrato es claro, la app puede crecer; si es improvisado, cada pantalla nueva aumenta la fragilidad del sistema.

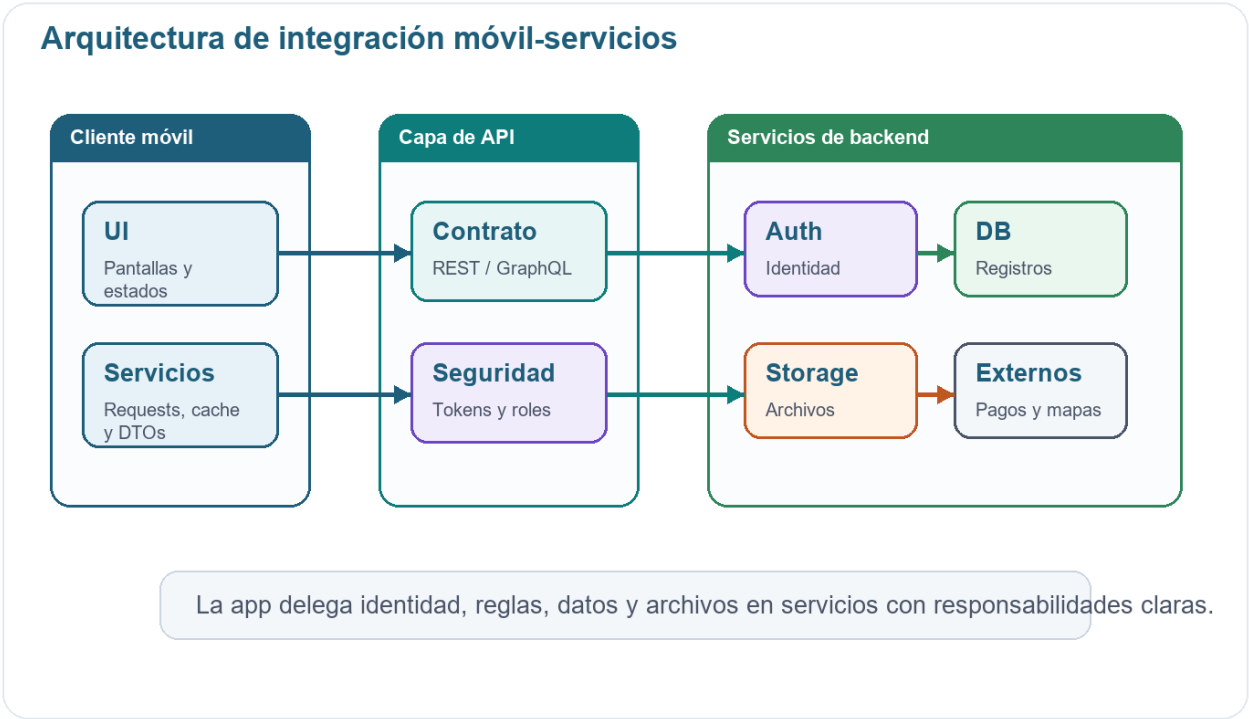


Figura 1. Vista general de una integración móvil con servicios de backend, autenticación, datos, almacenamiento y observabilidad.

1. La integración como decisión de arquitectura

En proyectos pequeños suele pensarse la integración al final, cuando la interfaz ya está lista. Ese enfoque produce problemas frecuentes: pantallas que necesitan datos que el backend no entrega, formularios que no coinciden con las reglas reales, errores difíciles de comunicar, duplicación de lógica entre cliente y servidor, o credenciales expuestas en el código.

Por eso, la integración debe diseñarse desde el inicio del vertical slice. Una porción funcional completa no solo atraviesa interfaz, navegación y estado; también atraviesa identidad, reglas de acceso, persistencia y manejo de errores.

1.1 Responsabilidades de la app y del backend

La app móvil debe ofrecer una experiencia rápida, clara y adaptada al contexto del usuario. El backend debe proteger datos, aplicar reglas de negocio, persistir información, controlar permisos y coordinar servicios externos. Cuando esas responsabilidades se mezclan, el sistema se vuelve inseguro y difícil de mantener.

Tabla 1. Separación de responsabilidades entre app móvil y backend.

Responsabilidad	En la app móvil	En el backend o servicio
Interfaz y experiencia	Presenta pantallas, estados, validaciones inmediatas y feedback.	Puede entregar configuraciones, textos o reglas para adaptar la experiencia.
Reglas de negocio	Puede validar formato y coherencia básica para mejorar la UX.	Debe validar siempre las reglas críticas antes de guardar o ejecutar acciones.
Seguridad	Protege tokens y evita exponer secretos.	Verifica identidad, permisos, cuotas, auditoría y políticas de acceso.
Datos	Cachea, transforma para la UI y maneja estados de carga/error.	Persiste, consulta, filtra, ordena, relaciona y protege datos.

Archivos	Selecciona, comprime, previsualiza y sube/descarga.	Controla rutas, permisos, metadatos, URLs firmadas y retención.
Observabilidad	Captura errores de cliente y contexto de uso.	Registra eventos, métricas, trazas y fallos de integración.

Regla práctica. La app puede ayudar al usuario a evitar errores, pero el backend debe ser la fuente de verdad para las reglas importantes. Toda validación crítica que exista solo en el cliente puede ser omitida o manipulada.

2. Estrategias de integración con backend

Una aplicación móvil puede integrarse con servicios de varias formas. La mejor alternativa depende del tipo de producto, el tiempo disponible, la experiencia del equipo, la sensibilidad de los datos y la necesidad de escalar. En la cátedra conviene priorizar soluciones que permitan construir una integración real y demostrable sin perder de vista las decisiones profesionales que habría que tomar en un proyecto productivo.

Tabla 2. Estrategias frecuentes de integración móvil con servicios.

Estrategia	Cuándo conviene	Ventajas	Riesgos o cuidados
Backend existente	La organización ya posee sistemas, APIs o bases de datos corporativas.	Aprovecha infraestructura real y reglas existentes.	Puede tener contratos viejos, documentación incompleta o restricciones de seguridad.
Backend propio	El equipo necesita reglas específicas y control completo.	Flexibilidad alta, dominio claro y evolución a medida.	Mayor costo de desarrollo, despliegue, monitoreo y seguridad.
BaaS: Supabase/Firebase/Appwrite	Proyectos educativos, MVPs o apps con autenticación, datos y storage estándar.	Acelera auth, base de datos, storage, reglas y SDKs.	Riesgo de acoplamiento a la plataforma y reglas mal configuradas.
Serverless / funciones	Tareas puntuales: procesar pagos, enviar notificaciones, transformar archivos.	Escala por demanda y reduce administración de servidores.	Dificulta flujos complejos si se abusa de funciones aisladas.
APIs de terceros	La app depende de mapas, pagos, clima, IA, mensajería o datos externos.	Permite incorporar capacidades maduras rápidamente.	Cambios de precio, límites de cuota, disponibilidad y privacidad.
No-code / low-code con APIs	Necesidad de validar una idea o automatizar procesos simples.	Velocidad y baja barrera de entrada.	Menor control del código y de algunas políticas de seguridad.

2.1 Criterios de decisión

- Sensibilidad de los datos: no es igual guardar preferencias que historias clínicas, pagos o documentos personales.
- Madurez del producto: un prototipo funcional puede apoyarse en BaaS; un producto crítico puede requerir backend propio o arquitectura híbrida.
- Capacidad del equipo: una solución sofisticada pero imposible de operar termina siendo más riesgosa que una solución simple bien configurada.
- Tiempo de entrega: para el trabajo práctico se recomienda priorizar un vertical slice pequeño, completo y demostrable.
- Evolución futura: conviene evitar decisiones que impidan crecer, auditar, migrar datos o cambiar proveedores.

3. Contratos de API y modelo de datos

Antes de escribir código cliente es necesario comprender el contrato de integración. Un contrato define qué operación existe, qué datos recibe, qué devuelve, qué errores puede producir y qué permisos exige. Puede estar documentado en OpenAPI/Swagger, en un schema GraphQL, en documentación propia del servicio o en ejemplos acordados por el equipo.

En móviles, el contrato debe pensarse con especial cuidado porque la red es variable, las sesiones pueden expirar, la app puede quedar suspendida y las pantallas tienen poco espacio para explicar fallos técnicos.

3.1 DTOs y transformación de datos

Un error común es usar directamente en la interfaz la estructura que llega desde la API. Es preferible transformar la respuesta en modelos más estables para la app. Estos objetos de transferencia de datos o DTOs ayudan a separar el contrato externo de la forma en que la pantalla necesita mostrar información.

```
// Ejemplo conceptual de transformación
respuesta API: { "first_name": "Ana", "last_name": "Paz", "avatar_url": null }
modelo UI:     { nombreCompleto: "Ana Paz", avatar: imagenPorDefecto, iniciales: "AP" }
```

- La API puede usar nombres, formatos o campos pensados para el backend.
- La pantalla necesita datos listos para mostrar, con valores por defecto y variantes visuales.
- La transformación permite manejar campos nulos, formatos de fecha, unidades, monedas o etiquetas.
- Si cambia el backend, el impacto queda concentrado en la capa de servicios o repositorios.

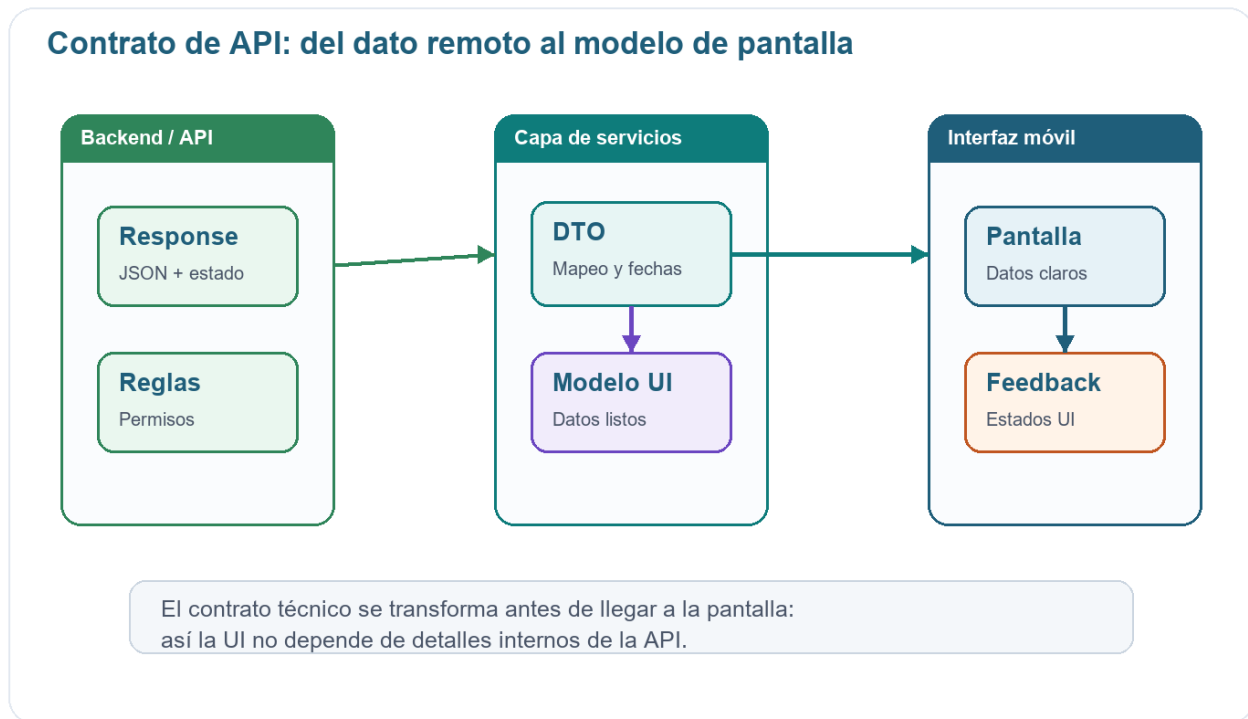


Figura 2. Transformación entre contrato de API, DTO, modelo de pantalla y estados visibles para el usuario.

3.2 Versionado y ambientes

Una integración profesional distingue ambientes: desarrollo, prueba y producción. También contempla versionado de API cuando un contrato puede cambiar sin romper versiones instaladas de la app. A diferencia de una web, una app móvil publicada puede permanecer instalada durante meses sin actualizarse; por eso el backend debe ser cuidadoso al eliminar campos o modificar respuestas.

4. Consumo de servicios RESTful

REST es un estilo de arquitectura muy utilizado para exponer recursos mediante HTTP. En una app móvil, consumir REST implica construir requests, enviar headers, interpretar códigos de estado, transformar JSON, mostrar estados de interfaz y recuperarse de errores.



Figura 3. Ciclo de consumo de un servicio REST desde una acción del usuario hasta la actualización de la interfaz.

4.1 Métodos HTTP y recursos

Tabla 3. Métodos HTTP comunes en integraciones RESTful.

Método	Uso típico	Ejemplo móvil
GET	Consultar recursos sin modificar el servidor.	Listar turnos, buscar productos, obtener perfil.
POST	Crear un recurso o ejecutar una operación.	Registrar usuario, crear reserva, iniciar pago.
PUT/PATCH	Actualizar un recurso existente.	Editar perfil, cambiar estado de una tarea.
DELETE	Eliminar o desactivar un recurso.	Quitar favorito, cancelar una inscripción.

4.2 Códigos de estado y decisiones de UI

Tabla 4. Códigos HTTP y comportamiento esperado en la interfaz.

Código	Significado	Respuesta recomendada en la app
200/201	Operación exitosa o recurso creado.	Mostrar resultado, actualizar estado y confirmar la acción.

400	Solicitud inválida.	Revisar datos ingresados y mostrar mensaje comprensible.
401	No autenticado.	Pedir iniciar sesión o renovar token si corresponde.
403	Sin permisos.	Explicar que la acción no está permitida; no repetir la solicitud.
404	Recurso inexistente.	Mostrar estado vacío o recurso no disponible.
409	Conflicto.	Indicar que el dato cambió y ofrecer recargar o resolver conflicto.
429	Demasiadas solicitudes.	Esperar, informar límite y evitar reintentos agresivos.
500/503	Error del servidor o servicio no disponible.	Mostrar error temporal, permitir reintentar y registrar diagnóstico.

4.3 Buenas prácticas de consumo REST en móviles

- Definir timeouts: una solicitud no debe quedar esperando indefinidamente.
- Evitar acciones duplicadas: deshabilitar botones mientras una operación crítica está en curso.
- Usar paginación o carga incremental para listas grandes.
- Representar estados de carga, vacío, error y éxito con componentes claros.
- No mostrar errores técnicos crudos al usuario; traducirlos a mensajes accionables.
- Centralizar la configuración base de la API: URL, headers, interceptores, serialización y manejo de sesión.
- Registrar errores con contexto suficiente para diagnosticar, sin exponer datos sensibles.

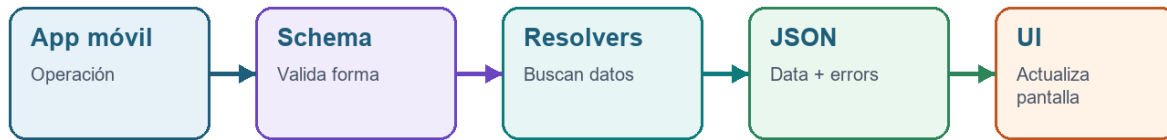
```
// Pseudocódigo de una llamada protegida
token = secureStore.get("access_token")
response = GET /api/turnos?page=1
headers: Authorization: Bearer <token>

si response.status == 200 -> mostrar datos
si response.status == 401 -> intentar renovar sesión o pedir login
si response.status >= 500 -> mostrar error temporal y permitir reintento
```

5. Consumo de GraphQL desde la app móvil

GraphQL permite que el cliente solicite exactamente la forma de datos que necesita mediante queries y mutations. En lugar de consumir muchos endpoints REST, la app envía una operación contra un endpoint GraphQL y recibe una respuesta con una estructura similar a la solicitada.

Flujo GraphQL: una operación, datos con forma precisa



```
query Perfil($id: ID!) { usuario(id: $id) { nombre avatarUrl turnos { fecha estado } } }
```

Figura 4. En GraphQL la app define una operación, el schema valida la forma posible y los resolvers obtienen los datos.

5.1 Ventajas y cuidados

Tabla 5. Comparación práctica entre REST y GraphQL para aplicaciones móviles.

Aspecto	REST	GraphQL
Modelo mental	Recursos y endpoints.	Schema, tipos, queries y mutations.
Cantidad de datos	Puede devolver más o menos de lo necesario.	La app pide campos específicos.
Evolución	Versionado por rutas o contratos.	Evolución del schema, deprecación de campos.
Errores	Códigos HTTP suelen guiar el flujo.	Puede haber data parcial y arreglo errors.
Caché	Aprovecha semántica HTTP con mayor facilidad.	Requiere estrategia de cliente o normalización.
Complejidad	Simple para operaciones CRUD claras.	Potente para pantallas con datos relacionados.

GraphQL resulta especialmente útil cuando una pantalla necesita combinar datos relacionados: perfil, últimas compras, notificaciones y permisos, por ejemplo. También puede reducir sobrecarga de red si evita múltiples requests. Sin embargo, exige disciplina: definir límites de profundidad, permisos por campo, manejo de caché y control de consultas costosas.

```
mutation CrearTurno($input: CrearTurnoInput!) {
  crearTurno(input: $input) { id fecha estado }
}
```


6. Autenticación y autorización

La autenticación responde quién es el usuario. La autorización responde qué puede hacer ese usuario. Confundir ambos conceptos lleva a errores graves: una persona puede haber iniciado sesión correctamente y, aun así, no tener permiso para ver un documento, editar un registro o descargar un archivo.



Figura 5. Flujo conceptual de autenticación, uso de tokens, renovación y autorización de recursos protegidos.

6.1 Flujos frecuentes de autenticación

Tabla 6. Flujos de autenticación habituales en aplicaciones móviles.

Flujo	Descripción	Uso recomendado
Email y contraseña	El usuario se registra e inicia sesión con credenciales propias.	Proyectos educativos, apps internas o MVPs con política de contraseña adecuada.
Magic link / OTP	El usuario recibe un enlace o código temporal por email/SMS.	Reducir fricción y evitar gestión de contraseña.
OAuth 2.0 / OpenID Connect	Inicio con proveedor externo o identidad corporativa.	Apps con Google, Apple, Microsoft, redes sociales o SSO institucional.
Biometría local	Huella o rostro desbloquean el acceso local a una sesión existente.	Mejorar comodidad, no reemplaza identidad del backend.

6.2 Tokens y ciclo de sesión

Muchas integraciones usan tokens. Un access token suele tener vida corta y se envía en cada request protegida. Un refresh token permite renovar la sesión sin obligar al usuario a iniciar sesión a cada momento. El manejo correcto de ambos evita interrupciones innecesarias y reduce el impacto si un token queda comprometido.

Tabla 7. Elementos del ciclo de sesión y cuidados básicos.

Elemento	Propósito	Cuidado principal
Access token	Autorizar requests protegidas durante un período breve.	No exponerlo en logs ni almacenarlo en texto plano.
Refresh token	Renovar la sesión cuando expira el access token.	Protegerlo especialmente y revocarlo al cerrar sesión.
Secure storage	Guardar tokens usando mecanismos del sistema operativo.	Usar Keychain, Keystore o SecureStore según plataforma/framework.
Logout	Cerrar sesión local y, si corresponde, invalidar tokens en backend.	Limpiar estado, caché sensible y navegación protegida.

Cuidado. No se deben incluir claves secretas, service keys ni credenciales administrativas dentro de la app móvil. Todo lo que se distribuye en una app puede ser inspeccionado. Las operaciones privilegiadas deben vivir en el backend.

7. Almacenamiento de datos y archivos en la nube

Además de consumir datos, muchas apps necesitan persistir información propia: perfiles, preferencias, formularios, historiales, imágenes, comprobantes o documentos. En integración móvil conviene distinguir datos estructurados de archivos binarios.

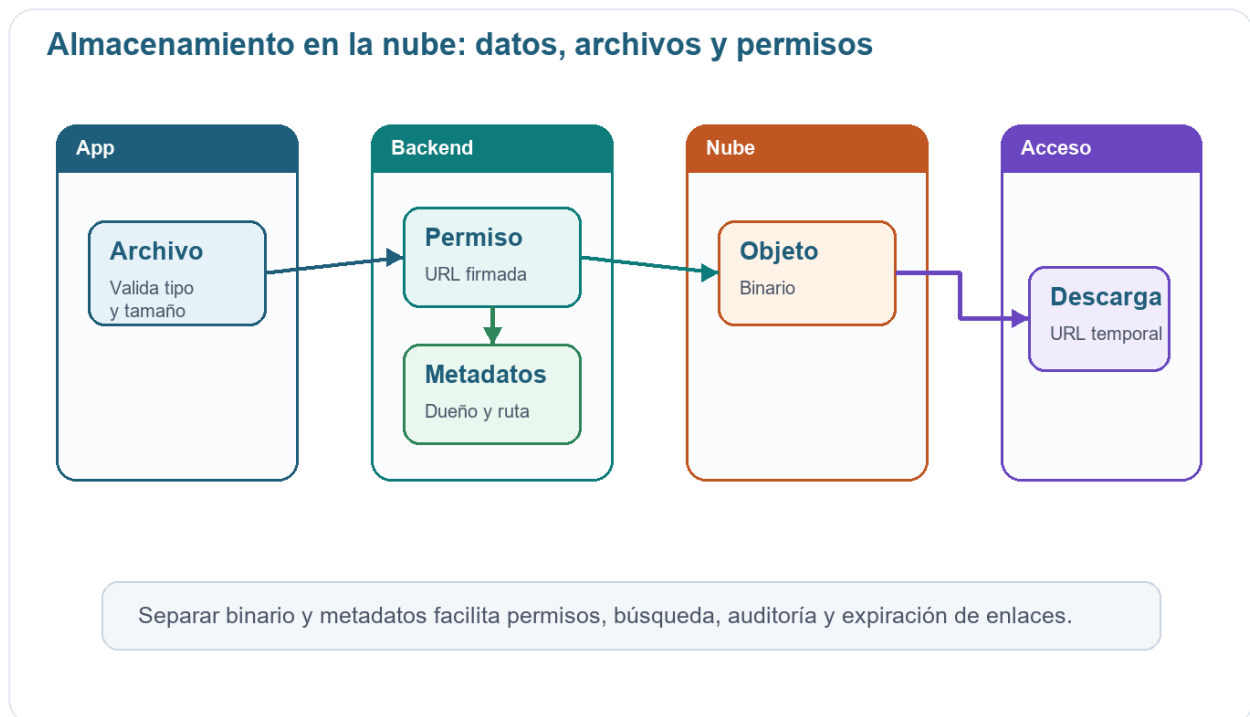


Figura 6. Flujo típico para subir archivos: la app valida, el backend autoriza, storage conserva el binario y la base guarda metadatos.

7.1 Datos estructurados

Los datos estructurados se guardan en tablas o colecciones: usuarios, productos, turnos, pedidos, comentarios. Requieren reglas de acceso claras. En plataformas como Supabase es común usar políticas a nivel de fila para que cada usuario vea solo los registros permitidos.

- Definir propietario del dato: qué usuario, equipo u organización controla cada registro.
- Evitar traer todos los campos si la pantalla solo necesita algunos.
- Aplicar filtros y paginación del lado del backend.
- Registrar fechas de creación y actualización para ordenar, auditar y sincronizar.
- Tratar borrados sensibles como desactivación lógica cuando se necesita historial.

7.2 Archivos y objetos

Los archivos no suelen guardarse dentro de la misma tabla que los datos. Se almacenan en object storage y se referencian mediante rutas o identificadores. La base de datos guarda metadatos: nombre visible, tipo, tamaño, dueño, fecha, estado de procesamiento y permisos.

Tabla 8. Opciones de almacenamiento según tipo de dato.

Tipo de contenido	Dónde conviene guardarlo	Ejemplos
Dato pequeño y consultable	Base de datos.	Nombre, email, estado, categoría, fecha.
Archivo binario	Object storage.	Avatar, foto, comprobante, PDF, audio.
Preferencia local no sensible	Almacenamiento local simple.	Tema, último filtro, orden elegido.
Dato sensible local	Secure storage o base local cifrada.	Token, claves de sesión, información personal mínima.
Datos offline complejos	Base local con estrategia de sincronización.	Formularios de campo, inspecciones, catálogos.

7.3 Sincronización y modo offline

No todas las apps necesitan funcionamiento offline completo, pero todas deberían manejar cortes de red de forma digna. Para una primera integración, alcanza con mostrar errores, permitir reintento y conservar estados no sensibles cuando sea útil. En productos más avanzados, la sincronización requiere resolver conflictos: qué ocurre si dos usuarios modifican el mismo dato, o si el usuario edita sin conexión y el servidor cambió mientras tanto.

8. Seguridad en la integración

La seguridad no es una capa final que se agrega después de programar. En integración móvil, cada decisión de arquitectura tiene consecuencias: dónde se guardan tokens, qué datos se envían, qué permisos tiene una clave, cómo se validan entradas, qué logs se conservan y cómo se responde ante errores.

Mapa de seguridad para una integración móvil

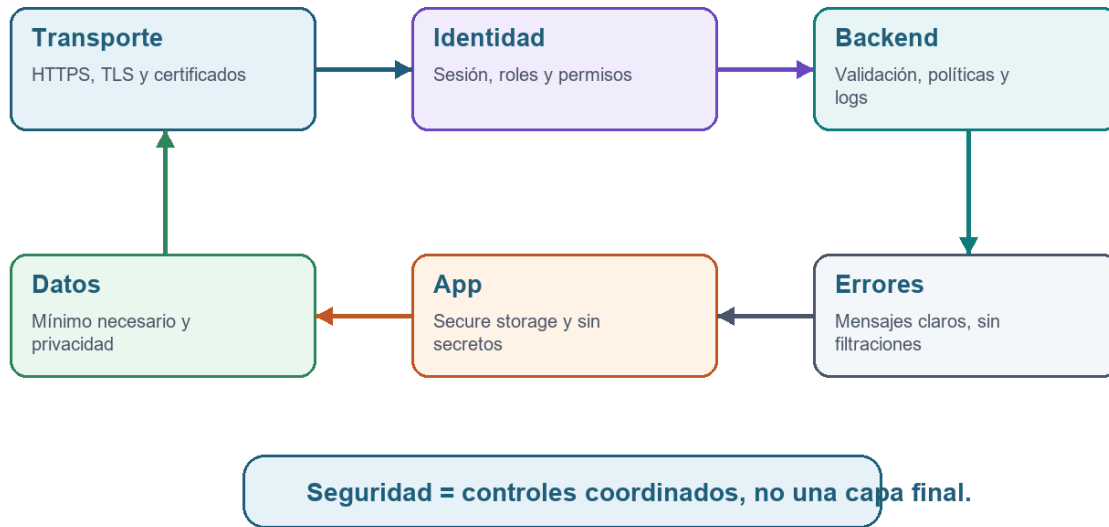


Figura 7. Mapa de aspectos de seguridad que deben revisarse en una integración móvil.

8.1 Principios básicos

- Mínimo privilegio: cada usuario, token y servicio debe tener solo los permisos necesarios.
- Validación en backend: todo dato recibido desde la app debe considerarse no confiable hasta ser validado.
- Transporte seguro: usar HTTPS; no enviar credenciales por canales inseguros.
- Protección de secretos: nunca guardar claves privadas o service keys en el repositorio ni en la app distribuida.
- Errores controlados: evitar respuestas que revelen estructura interna, SQL, rutas, claves o datos de otros usuarios.
- Datos mínimos: pedir y almacenar solo la información necesaria para la funcionalidad.
- Auditoría: registrar eventos relevantes de seguridad sin capturar datos sensibles innecesarios.

8.2 Riesgos frecuentes

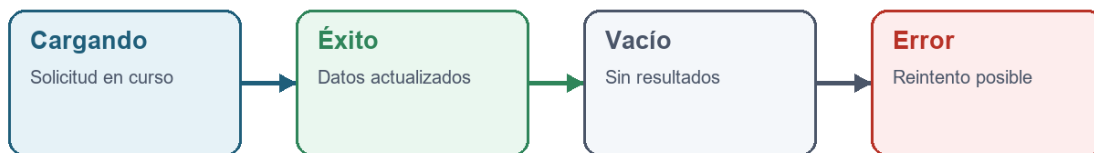
Tabla 9. Riesgos comunes de seguridad y medidas de prevención.

Riesgo	Ejemplo	Prevención
Autorización débil	Un usuario cambia un id en la URL y ve datos ajenos.	Verificar propietario y permisos en backend o políticas de datos.
Exposición excesiva	La API devuelve campos privados que la pantalla no muestra.	Diseñar respuestas mínimas y revisar DTOs.
Secretos hardcodeados	La app contiene una clave administrativa.	Usar backend para operaciones privilegiadas y variables de entorno seguras.
Tokens inseguros	El token queda en almacenamiento plano o logs.	Usar secure storage y limpiar logs.
Reintentos agresivos	La app satura el servicio ante fallos.	Backoff, límites y mensajes claros.
Validación solo cliente	El usuario manipula la app y envía datos inválidos.	Revalidar siempre en backend.

9. Observabilidad y manejo de errores

Una integración no se evalúa solo cuando funciona. También se evalúa cuando falla. Una app bien diseñada informa al usuario qué ocurrió, le ofrece una acción posible y deja evidencia técnica suficiente para que el equipo pueda investigar.

Estados de integración que la interfaz debe representar



Cada request debe terminar en un estado visible y comprensible:

la persona usuaria necesita saber si esperar, continuar, corregir o reintentar.

Figura 8. Estados que debería representar una interfaz móvil cuando se integra con servicios remotos.

Tabla 10. Estados de integración, experiencia de usuario y diagnóstico técnico.

Estado	Qué ve el usuario	Qué registra el equipo
Carga	Indicador visible, acción principal deshabilitada si corresponde.	Inicio de request, endpoint u operación, tiempo esperado.
Éxito	Confirmación, datos actualizados, navegación coherente.	Operación completada, duración, id de correlación si existe.
Vacío	Mensaje que explique ausencia de datos y acción siguiente.	Consulta válida sin resultados.
Error recuperable	Mensaje claro y botón de reintento.	Código, tipo de error, duración, conectividad aproximada.
Error de permiso	Explicación breve sin detalles internos.	Usuario, recurso, permiso requerido, política aplicada.

Criterio UX. El mensaje “algo salió mal” rara vez alcanza. Conviene decir qué puede hacer el usuario: revisar conexión, intentar nuevamente, iniciar sesión, actualizar datos o contactar soporte.

Bibliografía y recursos de referencia

- Tidwell, J.; Brewer, C.; Valencia, A. Designing Interfaces. O'Reilly Media, 2019.
- Dakić, M. Mobile App Development for Businesses: Create a Product Roadmap and Digitize Your Operations. Apress, 2023.
- Staiano, F. Designing and Prototyping Interfaces with Figma. Packt Publishing, 2023.
- Stimac, S. Design for Developers. Manning Publications, 2023.
- Conta, A. The Art and Science of UX Design. New Riders, 2023.
- Yablonski, J. Laws of UX. O'Reilly Media, 2024.
- Documentación oficial del framework utilizado por el equipo: Flutter, React Native, Ionic, Android, iOS u otra alternativa definida en el proyecto.
- Documentación oficial del servicio integrado: Supabase, Firebase, API REST, GraphQL, proveedor de autenticación o almacenamiento elegido.
- OWASP Mobile Application Security Verification Standard y guías de seguridad móvil como material complementario para revisar riesgos de integración.